AI ASSISTANT CODING

ASSIGNMENT-13

Name: B. Akhira Nandhini

Hallticket:2303A51516

Batch:22

Task Description #1 (Refactoring – Removing Global Variables)

• **Task:** Use AI to eliminate unnecessary global variables from the

code.

• **Instructions:**

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

rate = 0.1

def calculate_interest(amount):

return amount * rate

print(calculate_interest(1000))

code:

```
def calculate_interest(amount, rate):
    """
    Calculate the interest based on the given amount and interest rate.

    Parameters:
    amount (float): The principal amount for which interest is to be calculated.
    rate (float): The interest rate as a decimal (e.g., 0.1 for 10%).

    Returns:
    float: The calculated interest.
    """
    return amount * rate
# Take input from the user for the amount and interest rate
try:
    amount = float(input("Enter the principal amount: "))
    rate = float(input("Enter the interest rate (as a decimal, e.g., 0.1 for 10%): "))
    interest = calculate_interest(amount, rate)
    print(f"The calculated interest is: {interest}")
except ValueError:
    print("Please enter valid numerical values for amount and interest rate.")
```

• **Expected Output:**

o Refactored version passing rate as a parameter or using a configuration structure.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• **Task:** Use AI to refactor deeply nested if–elif–else logic into a

cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

```
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
```

```python
def evaluate_score(score):
    """
    Evaluate the score and return the corresponding performance category.

    Parameters:
    score (float): The score to be evaluated.

    Returns:
    str: The performance category based on the score.
    """
    if score >= 90:
        return "Excellent"
    if score >= 75:
        return "Very Good"
    if score >= 60:
        return "Good"
    return "Needs Improvement"
# Take input from the user for the score
try:
    score = float(input("Enter the score: "))
    result = evaluate_score(score)
    print(result)
except ValueError:
    print("Please enter a valid numerical value for the score.")
```

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

Task 3 (Refactoring Repeated File Handling Code)

• **Task:** Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

f = open("data1.txt")

print(f.read())

f.close()

f = open("data2.txt")

print(f.read())

f.close()

```python
def read_and_print_file(filename):
    """
    Read the contents of a file and print it to the console.

    Parameters:
    filename (str): The name of the file to be read.

    Returns:
    None
    """
    try:
        with open(filename, 'r') as file:
            content = file.read()
            print(content)
    except FileNotFoundError:
        print(f"The file '{filename}' was not found.")
    except IOError:
        print(f"An error occurred while trying to read the file '{filename}'.")
# Take input from the user for the filename
filename = input("Enter the filename to read: ")
read_and_print_file(filename)
```

```
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the filename to read: data.txt
Hello , My name is Adepu Tejaswini
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the filename to read: data1.txt
This is My AIAC Assignment-13.5
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the filename to read: data2.txt
The file 'data2.txt' was not found.
```

Expected Outcome:

o Reusable function using with open() and parameters.

Task 4 (Optimizing Search Logic)

• **Task:** Refactor inefficient linear searches using appropriate

data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

```
def check_user_access(username, user_set):
    """
    Check if the given username exists in the set of users and return access status.

    Parameters:
    username (str): The username to be checked.
    user_set (set): A set containing valid usernames.

    Returns:
    str: "Access Granted" if the username is found, otherwise "Access Denied".

    Time Complexity: O(1) for average case due to set lookup.
    """
    return "Access Granted" if username in user_set else "Access Denied"
# Create a set of valid usernames for efficient lookup
valid_users = {"admin", "guest", "editor", "viewer"}
# Take input from the user for the username
username = input("Enter username: ")
access_status = check_user_access(username, valid_users)
print(access_status)
```

```
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter username: admin
Access Granted
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter username: Tejaswini
Access Denied
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter username: manager
Access Denied
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter username: guest
Access Granted
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter username: viewer
Access Granted
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Task 5 (Refactoring Procedural Code into OOP Design)

• **Task:** Use AI to refactor procedural code into a class-based

design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

```python
class EmployeeSalaryCalculator:
    """
    A class to calculate the net salary of an employee after applying tax.

    Attributes:
    salary (float): The gross salary of the employee.
    tax_rate (float): The tax rate to be applied as a decimal (e.g., 0.2 for 20%).

    Methods:
    calculate_tax(): Calculates the tax amount based on the salary and tax rate.
    calculate_net_salary(): Calculates the net salary after deducting the tax from the gross salary.
    """

    def __init__(self, salary, tax_rate):
        """
        Initializes the EmployeeSalaryCalculator with the given salary and tax rate.

        Parameters:
        salary (float): The gross salary of the employee.
        tax_rate (float): The tax rate to be applied as a decimal (e.g., 0.2 for 20%).
        """
        self.salary = salary
        self.tax_rate = tax_rate

    def calculate_tax(self):
        """
        Calculate the tax amount based on the salary and tax rate.

        Returns:
        float: The calculated tax amount.
        """
        return self.salary * self.tax_rate

    def calculate_net_salary(self):
        """
        Calculate the net salary after deducting the tax from the gross salary.

        Returns:
        float: The calculated net salary.
        """
        tax_amount = self.calculate_tax()
        return self.salary - tax_amount
# Take input from the user for salary and tax rate
try:
    salary = float(input("Enter the gross salary: "))
    tax_rate = float(input("Enter the tax rate (as a decimal, e.g., 0.2 for 20%): "))
    calculator = EmployeeSalaryCalculator(salary, tax_rate)
    net_salary = calculator.calculate_net_salary()
    print(f"The net salary is: {net_salary}")
except ValueError:
    print("Please enter valid numerical values for salary and tax rate.")
```

```
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the gross salary: 50000
Enter the tax rate (as a decimal, e.g., 0.2 for 20%): 0.2
The net salary is: 40000.0
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the gross salary: 600000
Enter the tax rate (as a decimal, e.g., 0.2 for 20%): 0.1
The net salary is: 540000.0
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the gross salary: 45000
Enter the tax rate (as a decimal, e.g., 0.2 for 20%): 0.15
The net salary is: 38250.0
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Task 6 (Refactoring for Performance Optimization)

• **Task:** Use AI to refactor a performance-heavy loop handling

large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

```
total = 0

for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)
```

```python
import time
def sum_of_even_numbers(limit):
    """
    Calculate the sum of even numbers up to a given limit using a mathematical formula.

    Parameters:
    limit (int): The upper limit up to which even numbers are to be summed.

    Returns:
    int: The sum of even numbers up to the given limit.

    Time Complexity: O(1) due to direct calculation using the formula.
    """
    # Calculate the number of even numbers up to the limit
    n = limit // 2
    # Use the formula for the sum of the first n even numbers: n * (n + 1)
    return n * (n + 1)
# Take input from the user for the limit
try:
    limit = int(input("Enter the upper limit: "))
    start_time = time.time()
    result = sum_of_even_numbers(limit)
    end_time = time.time()
    print(f"The sum of even numbers up to {limit} is: {result}")
    print(f"Execution time: {end_time - start_time:.6f} seconds")
except ValueError:
    print("Please enter a valid integer for the limit.")
```

```
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the upper limit: 10
The sum of even numbers up to 10 is: 30
Execution time: 0.000000 seconds
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the upper limit: 0
The sum of even numbers up to 0 is: 0
Execution time: 0.000000 seconds
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the upper limit: 100
The sum of even numbers up to 100 is: 2550
Execution time: 0.000000 seconds
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter the upper limit: 1000000
The sum of even numbers up to 1000000 is: 250000500000
Execution time: 0.000000 seconds
```

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Task 7 (Removing Hidden Side Effects)

• **Task:** Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

data = []

def add_item(x):

data.append(x)

add_item(10)

add_item(20)

print(data)

```python
def add_item_to_list(item, data_list):
    """
    Add an item to a list and return the new list without modifying the original list.

    Parameters:
    item: The item to be added to the list.
    data_list (list): The original list to which the item will be added.

    Returns:
    list: A new list containing all items from the original list plus the new item.
    """
    # Create a new list by concatenating the original list with the new item
    return data_list + [item]
# Take input from the user for the item to be added
try:
    item = input("Enter an item to add to the list: ")
    original_list = []  # Original list is empty
    new_list = add_item_to_list(item, original_list)
    print(f"Original list: {original_list}")
    print(f"New list after adding item: {new_list}")
except Exception as e:
    print(f"An error occurred: {e}")
```

```
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter an item to add to the list: 10,20
Original list: []
New list after adding item: ['10,20']
```

Expected Outcome:

o Refactored function returning values instead of mutating globals.

Task 8 (Refactoring Complex Input Validation Logic)

- **Task:** Use AI to simplify and modularize complex validation rules.
- **Focus Areas:**

o Readability

o Testability

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

```python
def check_length(password):
    """
    Check if the password meets the minimum length requirement.

    Parameters:
    password (str): The password to be checked.

    Returns:
    bool: True if the password is at least 8 characters long, False otherwise.
    """
    return len(password) >= 8
def check_digit(password):
    """
    Check if the password contains at least one digit.
    Parameters:
    password (str): The password to be checked.
    Returns:
    bool: True if the password contains at least one digit, False otherwise.
    """
    return any(c.isdigit() for c in password)
def check_uppercase(password):
    """
    Check if the password contains at least one uppercase letter.
    Parameters:
    password (str): The password to be checked.
    Returns:
    bool: True if the password contains at least one uppercase letter, False otherwise.
    """
    return any(c.isupper() for c in password)
def validate_password(password):
    """
    Validate the password against multiple criteria and print appropriate messages.
    Parameters:
    password (str): The password to be validated.
    Returns:
    None
    """
    if not check_length(password):
        print("Password too short")
        return
    if not check_digit(password):
        print("Must contain digit")
        return
    if not check_uppercase(password):
        print("Must contain uppercase")
        return
    print("Valid Password")
# Take input from the user for the password
password = input("Enter password: ")
validate_password(password)
```

```
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter password: Abc12345
Valid Password
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter password: abc123
Password too short
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter password: ABCDEFG
Password too short
PS C:\Users\AdepuTejaswini\AI assist> python -u "c:\Users\AdepuTejaswini\AI assist\code-13.py"
Enter password: Password1
Valid Password
```

Expected Outcome:

o Separate validation functions with clear responsibility